



Department of Computer Science
Subject: Operating System

Submitted to: Sir Nasir

Submitted by:Eman Tariq

Reg. number: 23-NTU-CS-1150

Hometask

Semester:5th

Operating Systems – COC 3071

SE 5th A – Fall 2025

After-mid Homework -1

Part 1: Semaphore theory

1. A counting semaphore is initialized to 7. If 10 wait() and 4 signal() operations are performed, find the final value of the semaphore.

Q1: Initial $S=7$
 $10 \text{ wait}() \rightarrow 7-10 = -3 \Rightarrow 3 \text{ processes}$
 $\text{are waiting outside (no space)}$
 $4 \text{ signal}() \rightarrow -3+4 \rightarrow 1 \text{ (have space for 1 more process)}$
So final semaphore value = 1

2. A semaphore starts with value 3. If 5 wait() and 6 signal() operations occur, calculate the resulting semaphore value.

Q2: Initial $S=3$
 $5 \text{ wait}() = 3-5 = -2$
 $6 \text{ signal}() = -2+6 = 4$
So final semaphore value = 4

3. A semaphore is initialized to 0. If 8 signal() followed by 3 wait() operations are executed, find the final value.

Q3: $S=0$
 $8 \text{ signal}() \rightarrow 0+8 = 8 \text{ (8 spaces free)}$
 $3 \text{ wait}() \rightarrow 8-3 = 5$
Final semaphore value = 5

4. A semaphore is initialized to 2. If 5 wait() operations are executed:
a) How many processes enter the critical section?
b) How many processes are blocked?

Q4: $S \rightarrow 2$
 $5 \text{ wait}() \rightarrow 2-5 = -3$
a) 2 processes entered the critical section and 3 are still waiting.
b) 3 processes are blocked.

5. A semaphore starts at 1. If 3 wait() and 1 signal() operations are performed:

a) How many processes remain blocked?

b) What is the final semaphore value?

Q5: $s = 1$
3 are waiting so, $1 - 3 = -2$ (2 processes blocked)
now 2 signals are generated so,
 $-2 + 1 = -1$ (1 remains blocked)
a) 1 process remain blocked.
b) final semaphore value is $s = -1$.

6.

semaphore $S = 3$;

wait(S);

wait(S);

signal(S);

wait(S);

wait(S);

a) How many processes enter the critical section?

b) What is the final value of S?

Q6: $S = 3$
wait(S) perform wait operation on S
that is $s = s - 1$
So,
• $\text{wait}(S) = 3 - 1 = 2$ (1 process went into C.S.)
• $\text{wait}(S) = 2 - 1 = 1$ (1 process into the C.S.)
• $\text{Signal}(S) = 1 + 1 = 2$ (Applying signal operation $s = s + 1$) $\rightarrow$ 2 spaces free (no process went inside at this point)
• $\text{wait}(S) = 2 - 1 = 1$ (1 process inside C.S.)
• $\text{wait}(S) = 1 - 1 = 0$ (1 process went inside)
• process that entered critical section = 4
• Final value of $s = 0$

7.

semaphore $S = 1$;

wait(S);

wait(S);

signal(S);

signal(S);

a) How many processes are blocked?

b) What is the final value of S?

⑦ $S = 1$

$\text{wait}(S) = 1 - 1 = 0$ (1 process into C.S.)

$\text{wait}(S) = 0 - 1 = -1$ (process blocked)

$\text{signal}(S) = -1 + 1 = 0$ (1 process out)
(1 wakeup)

$\text{signal}(S) = 0 + 1 = 1$ (1 space free)

- process blocked = 0
- End $S = 1$

8. A binary semaphore is initialized to 1. Five wait() operations are executed without any signal(). How many processes enter the critical section and how many are blocked?

⑧ $S = 1$

5 $\text{wait}() = 1 - 5 = -4$ (4 processes blocked, 1 went into critical section)

- 4 blocked
- 1 enters the critical section

9. A counting semaphore is initialized to 4. If 6 processes execute wait() simultaneously, how many proceed and how many are blocked?

⑨ $S = 4$

6 $\text{wait}() = 4 - 6 = -2$ (2 processes blocked, rest into the critical section).

10. A semaphore S is initialized to 2. wait(S); wait(S); wait(S); signal(S); signal(S); wait(S);

- a) Track the semaphore value after each operation.
b) How many processes were blocked at any time?

⑩ $S=2$
 $\text{wait}() = S = S - 1$
 $S = 2 - 1 = 1 \rightarrow (1 \text{ process into C.S.})$
 $\text{wait}(S) = 1 - 1 = 0$ (Another process into C.S.)
 $S = 0 \rightarrow (\text{no space available})$
 $\text{wait}(S) = 0 - 1 = -1$ (1 process blocked)
 $\text{signal}(S) = -1 + 1 = 0$ (1 process wakes up)
 $\text{signal}(S) = 0 + 1 = 1$ (1 space available)
 $\text{wait}(S) = 1 - 1 = 0$ (1 process into C.S.)
One process was blocked.

11. A semaphore is initialized to 0. Three processes execute wait() before any signal(). Later, 5 signal() operations are executed. a) How many processes wake up?
b) What is the final semaphore value?

⑪ $S=0$
3 $\text{wait}() = 0 - 3 = -3$ (All three blocked)
5 $\text{signal} = -3 + 5 = 2$ (3 processes into the critical section, 2 spaces still available)
a) 3 processes woke up
b) $S=2$

Part 2: Semaphore Coding

Consider the Producer–Consumer problem using semaphores as implemented in Lab-10 (Lab-plan attached). Rewrite the program in your own coding style, compile and execute it successfully, and explain the working of the code in your own words.

Submission Requirements:

- Your rewritten source code
- A brief description of how the code works
- Screenshots of the program output showing successful execution

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    int id = *(int*)arg;

    for(int i = 0; i < 3; i++) {
        int item = id * 100 + i;

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer %d produced %d at %d\n", id, item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    return NULL;
}

void* consumer(void* arg) {
    int id = *(int*)arg;

    for(int i = 0; i < 3; i++) {
        sem_wait(&full);
```



```

        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumer %d consumed %d from %d\n", id, item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(2);
        return NULL;
    }
}

int main() {
    pthread_t prod[2], cons[2];
    int ids[2] = {1, 2};

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    for(int i = 0; i < 2; i++) {
        pthread_create(&prod[i], NULL, producer, &ids[i]);
        pthread_create(&cons[i], NULL, consumer, &ids[i]);
    }
    for(int i = 0; i < 2; i++) {
        pthread_join(prod[i], NULL);
        pthread_join(cons[i], NULL);
    }
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("All done!\n");
    return 0;
}

```

Terminal:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
eman@DESKTOP-ITNHL9J:~/Operating-System/lab10$ gcc hometask.c -o hometask -lpthread
eman@DESKTOP-ITNHL9J:~/Operating-System/lab10$ ./hometask
Producer 1 produced 100 at 0
Producer 2 produced 200 at 1
Producer 1 produced 101 at 2
Producer 2 produced 201 at 3
Producer 1 produced 102 at 4

```

How It Works:

Buffer:

- A small array where producers put items and consumers take items.
- in tells where the next item goes, out tells where the next item is taken from.

Producer:

- Waits if the buffer is full.
- Locks the buffer to safely put an item.
- Unlocks the buffer and tells consumers that there's a new item.

Consumer:

- Waits if the buffer is empty.
- Locks the buffer to safely take an item.
- Unlocks the buffer and tells producers that there's space.

Synchronization:

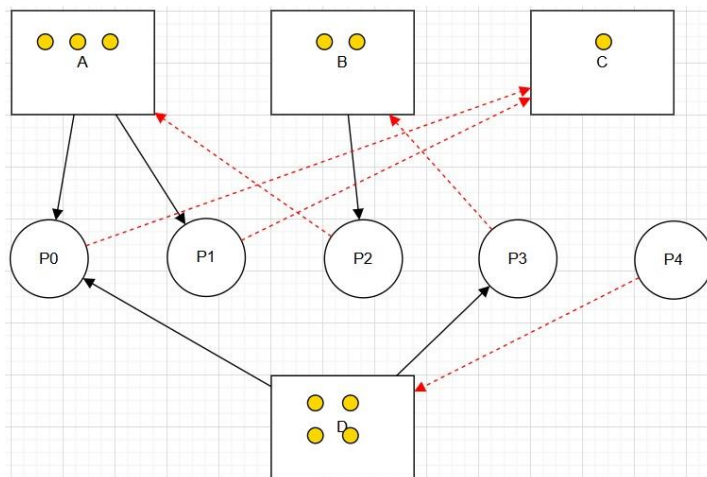
- Semaphore empty -> Counts empty spaces.
- Semaphore full -> Counts items in the buffer.
- Mutex -> Ensures only one thread touches the buffer at a time.

Circular Buffer:

When in or out reach the end of the array, they go back to the start.

Part 3: RAG (Recourse Allocation Graph)

- Convert the following graph into matrix table ,



Part 3 : RAQ

Process to resource $\rightarrow$ Request
Resource to Process $\rightarrow$ Holding (Allocated)

Now,

2 possible matrices,

- ① Requested Matrix
- ② Allocation matrix

Allocation:

Process	A	B	C	D
P ₀	1	0	0	0
P ₁	1	0	0	0
P ₂	0	1	0	0
P ₃	0	0	0	1
P ₄	0	0	0	0

② Request:

Process	A	B	C	D
P ₀	0	0	1	0
P ₁	0	0	1	0
P ₂	1	0	0	0
P ₃	0	1	0	0
P ₄	0	0	0	1

Part 4: Banker's Algorithm

System Description:

- The system comprises four processes (P₀–P₃) and four resources (A,B,C,D).
- Total Existing Resources:

Total			
A	B	C	D
6	4	4	2

- Snapshot at the initial time stage:

	Allocation				Max				Need			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	1	1	3	2	1	1				
P1	1	1	0	0	1	2	0	2				
P2	1	0	1	0	3	2	1	0				
P3	0	1	0	1	2	1	0	1				

.

Questions:

1. Compute the Available Vector:

- Calculate the available resources for each type of resource.

Total Allocation of each resource,

$$A = P0(2) + P1(1) + P2(1) + P3(0) = 4$$

$$B = P0(0) + P1(1) + P2(0) + P3(1) = 2$$

$$C = P0(1) + P1(0) + P2(1) + P3(0) = 2$$

$$D = P0(1) + P1(0) + P2(0) + P3(1) = 2$$

Now Available value of each resource = Total - Allocated

$$A = 6 - 4 = 2$$

$$B = 4 - 2 = 2$$

$$C = 4 - 2 = 2$$

$$D = 2 - 2 = 0$$

Available Vector=(2,2,2,0)

2. Compute the Need Matrix:

- Determine the need matrix by subtracting the allocation matrix from the maximum matrix.

Need(Max - Allocated):

	A	B	C	D
P0	1	2	0	0
P1	0	1	0	2
P2	2	2	0	0

P3	2	0	0	0
-----------	---	---	---	---

3.Safety Check:

- Determine if the current allocation state is safe. If so, provide a safe sequence of the processes.
- Show how the Available (working array) changes as each process terminates.

Answer:

Safe when Need $\leq$ Available

Initially available vector=(**2,2,2,0**)

For P0:

Need(1,2,0,0) $\leq$ Available(2,2,2,0)

thus P0 process can be completed

Now once completed,

Available vector= Previously Available+ AllocationP0

$$=(2,2,2,0)+(2,0,1,1)=(4,2,3,1)$$

For P1:

Need(0,1,0,2) $\leq$ Available(4,2,3,1) (X)

thus P1 process cant be completed

We need D(2) available=1

THUS THE GIVEN SEQUENCE IS NOT FEASIBLE THE CORRECT SEQUENCE IS,

Safe when Need $\leq$ Available

Initially available vector=(**2,2,2,0**)

For P0:

Need(1,2,0,0) $\leq$ Available(2,2,2,0)

thus P0 process can be completed

Now once completed,

Available vector= Previously Available+ AllocationP0

$$=(2,2,2,0)+(2,0,1,1)=(4,2,3,1)$$

For P2:

Need(2,2,0,0) $\leq$ Available(4,2,3,1)

thus P2 process completed

Now once completed,

Available vector= Previously Available+ AllocationP2

$$=(4,2,3,1)+(1,0,1,0)=(5,2,4,1)$$

For P3:

Need(2,0,0,0) $\leq$ Available(5,2,4,1)

thus P3 process completed

Now once completed,

Available vector= Previously Available+ AllocationP2

$$=(5,2,4,1)+(0,1,0,1)=(5,3,4,2)$$

For P1:

Need(0,1,0,2) $\leq$ Available(5,3,4,2)

thus P1 process completed

Now once completed,

Available vector= Previously Available+ AllocationP2

$$=(5,3,4,2)+(1,1,0,0)=(6,4,4,2)$$

Thus the system is in safe state

P0->P2->P3->P1